

班級：資訊三乙 學號：D1210799 姓名：王建葦

一、【實驗目的】：

What was your design? What were the concepts you have used for your design?

本次實驗目的為熟悉 **Timer 計時器** 與 **中斷 (Interrupt)** 的使用方式，並將其應用於 LCD 動畫播放與七段顯示器同步計時，實作「小綠人 (Little Green Man)」行走動畫系統。

Lab10-1 — 小綠人 I (基礎版)

設計一個小綠人動畫播放系統：

- 小綠人由 **6 張 64×64 bitmap** 動作影格組成，依序循環播放。
- 使用 **Timer** 控制動畫更新時間，**預設每 1 秒**切換一個動作。
- 七段顯示器顯示小綠人行走的「累計秒數」。
- 按下 **S 鍵 (5)**：
 - 第一次：動畫停止，七段顯示器歸零
 - 第二次：動畫重新開始，重新計時

此實驗主要練習：

- Timer 週期性中斷
- LCD bitmap 顯示
- 七段顯示器同步計時

- 動畫播放狀態控制
-

Lab10-2 — 小綠人 II (進階版)

在 Lab10-1 基礎上，加入 **動畫速度控制功能**：

- 保留原本 Start / Stop 功能 (S 鍵) 。
- 新增速度切換：
 - ← 鍵 (4) : 減速
 - 鍵 (6) : 加速
- 動畫速度共有 **4 段**：
 - 2 秒 / 動作
 - 1 秒 / 動作
 - 0.5 秒 / 動作
 - 0.25 秒 / 動作
- Timer 依目前速度設定調整比較值 (CMP)，即時影響動畫播放速度。

本實驗整合：

- **Timer 參數動態調整**
- **狀態機 (Running / Stop)**
- **人機介面 (Keypad 控制)**
- **動畫與計時同步顯示**

二、【遭遇的問題】：

What problems you faced during design and implementation?

- **Timer** 初期設定不正確，導致動畫播放速度異常（過快或不穩定）。
- 七段顯示器若與動畫更新在同一主迴圈，容易閃爍。
- 按鍵長按時會重複觸發，造成動畫不停啟動或速度狂跳。
- 動畫停止後，**Timer** 若未正確重置，重新開始時計時不準。
- Lab10-2 中切換速度時，動畫會瞬間跳格或延遲。

三、【解決方法】：

How did you solve the problems?

- 使用 **Timer 週期性中斷** 來控制動畫影格切換，避免以軟體 delay 造成不穩定。
- 將七段顯示器更新與動畫播放解耦：
 - **Timer** 負責秒數累計
 - 主程式僅處理狀態切換
- 加入 **Key Release Detection**（按鍵放開偵測），避免長按造成重複觸發。
- 在按下 **S** 鍵停止動畫時：
 - 清除計時變數
 - 將七段顯示歸零
 - 重設動畫索引
- 在 Lab10-2 中，使用不同的 **Timer 比較值（CMP）** 對應不同速度，使速度切換平滑且即時。

在找尋這些問題的解決方法與問題點時，我有使用 ChatGPT 協助我找尋與解決問題。包含 實驗結報的內容修改與潤飾都有使用 ChatGPT 協助。

四、【未能解決的問題】：

Was there any problem that you were unable to solve? Why was it unsolvable?

1. LCD 為單緩衝顯示，動畫切換時仍有極輕微閃爍，需雙緩衝架構才能完全解決。
2. 七段顯示器僅顯示秒數，無法顯示目前動畫速度等狀態資訊。
3. 若頻繁快速切換速度，動畫仍可能出現短暫不同步現象。
4. Bitmap 為固定大小，無法動態縮放或旋轉，動畫變化有限。

五、【程式碼】：

Lab10-1:

[illegible]

[illegible]


```

32 // ===== 2. 全域變數定義 =====
33 // 動畫控制變數 (使用 volatile 關鍵字，避免編譯器優化)
34 volatile uint32_t animation_counter = 0; // 動畫計數器 (用於控制動畫切換速度)
35 volatile uint32_t time_counter = 0;      // 時間計數器 (記錄動畫播放時間，單位：0.01秒)
36 volatile uint8_t current_frame = 0;      // 當前動畫幀索引 (0-5)
37 volatile uint8_t is_running = 1;         // 動畫運行狀態標誌 (1=運行，0=停止)
38 volatile uint8_t speed_index = 1;        // 速度索引 (0-3，對應不同速度等級)
39 volatile uint8_t lcd_update_flag = 0;    // LCD 更新標誌 (1=需要更新，0=無需更新)
40
41 // 按鍵處理變數
42 volatile uint8_t key_buffer = 0; // 按鍵緩衝區 (儲存按下的按鍵值)
43 volatile uint8_t key_lock = 0;    // 按鍵鎖定標誌 (防止重複觸發)
44
45 // 七段顯示器變數
46 volatile uint8_t seg_digit[4] = {0, 0, 0, 0}; // 七段顯示器四位數字陣列
47
48 // 速度設定陣列 (對應 speed_index 0-3)
49 // 數值越大，動畫切換越慢 (需要更多次 Timer0 中斷才切換一幀)
50 const uint32_t speed_settings[4] = {200, 100, 50, 25};
51
52 // 動畫幀指標陣列 (指向 6 個綠色小人點陣圖)
53 const unsigned char *frames[6];
54
55 // ===== 3. 函數宣告 =====
56 void Init_Timer0(void);
57 void Init_Timer1(void);
58 void Update_Animation(void);
59 void Process_Keys(void);

```

```

61 // ===== 4. Timer0 中斷處理函數 (100Hz = 每 10ms 觸發一次) =====
62 /**
63  * @brief Timer0 中斷服務程式
64  * @details 負責控制動畫播放速度和時間計數
65  *          - 每 10ms 觸發一次 (100Hz)
66  *          - 根據速度設定決定何時切換動畫幀
67  *          - 更新時間計數器和七段顯示器數字
68  */
69 void TMR0_IRQHandler(void)
70 {
71     // 區域變數宣告 (C89 標準要求所有變數宣告必須放在函數開頭)
72     uint32_t total_seconds; // 總秒數 (用於計算七段顯示器的顯示值)
73
74     // 只有在動畫運行狀態下才執行以下操作
75     if (is_running)
76     {
77         // ===== 1. 動畫控制邏輯 =====
78         animation_counter++; // 動畫計數器遞增 (每次中斷 +1)
79
80         // 檢查是否達到當前速度設定值 (需要多少次中斷才切換一幀)
81         if (animation_counter >= speed_settings[speed_index])
82         {
83             animation_counter = 0; // 重置動畫計數器
84             current_frame = (current_frame + 1) % 6; // 切換到下一幀 (循環 0-5)
85             lcd_update_flag = 1; // 設定 LCD 更新標誌, 通知主程式更新畫面
86         }
87
88         // ===== 2. 時間計數 =====
89         time_counter++; // 時間計數器遞增 (每 10ms +1, 單位: 0.01秒)
90
91         // ===== 3. 數值轉換 (將時間計數器轉換為七段顯示器數字) =====
92         // 將時間計數器 (單位: 0.01秒) 轉換為總秒數
93         // 例如: time_counter = 1234 → total_seconds = 12.34 秒 (取整數部分)
94         total_seconds = time_counter / 100;
95
96         // 將總秒數分解為四位數字, 存入七段顯示器陣列
97         // seg_digit[0]: 個位數 (0-9)
98         seg_digit[0] = total_seconds % 10;
99         // seg_digit[1]: 十位數 (0-9)
100        seg_digit[1] = (total_seconds / 10) % 10;
101        // seg_digit[2]: 百位數 (0-9)
102        seg_digit[2] = (total_seconds / 100) % 10;
103        // seg_digit[3]: 千位數 (0-9)
104        seg_digit[3] = (total_seconds / 1000) % 10;
105    }
106
107    // 清除 Timer0 中斷標誌 (必須執行, 否則會持續觸發中斷)
108    TIMER_ClearIntFlag(TIMER0);
109 }

```

```

111 // ===== 5. Timer1 中斷處理函數 (1kHz = 每 1ms 觸發一次) =====
112 /**
113  * @brief Timer1 中斷服務程式
114  * @details 負責兩個任務：
115  *           1. 七段顯示器多工掃描 (每秒掃描 1000 次，每個數字 250 次)
116  *           2. 按鍵掃描和防彈跳處理 (每 20ms 掃描一次)
117  */
118 void TMR1_IRQHandler(void)
119 {
120     // 使用 static 變數保持狀態 (符合 C89 標準)
121     static uint8_t scan_index = 0; // 七段顯示器掃描索引 (0-3)
122     static uint8_t key_check_counter = 0; // 按鍵掃描計數器
123     uint8_t temp_key;
124
125     // ===== 任務 1: 七段顯示器多工掃描 =====
126     // 關閉所有七段顯示器 (避免鬼影)
127     CloseSevenSegment();
128     // 顯示當前掃描位置的數字
129     ShowSevenSegment(scan_index, seg_digit[scan_index]);
130     // 移動到下一個位置
131     scan_index++;
132     if (scan_index > 3)
133         scan_index = 0; // 循環掃描 0-3
134
135     // ===== 任務 2: 按鍵掃描 (防彈跳處理) =====
136     key_check_counter++;
137     if (key_check_counter >= 20)
138     { // 每 20ms 掃描一次按鍵 (50Hz 掃描頻率)
139         key_check_counter = 0;
140
141         temp_key = ScanKey(); // 讀取按鍵值
142
143         if (temp_key != 0)
144         {
145             // 偵測到按鍵按下
146             if (key_lock == 0)
147             {
148                 // 如果按鍵未鎖定，儲存按鍵值並鎖定
149                 key_buffer = temp_key; // 將按鍵值存入緩衝區
150                 key_lock = 1; // 鎖定按鍵，防止重複觸發
151             }
152         }
153         else
154         {
155             // 當 ScanKey 返回 0 (無按鍵按下)，解除鎖定
156             key_lock = 0;
157         }
158     }
159
160     TIMER_ClearIntFlag(TIMER1); // 清除 Timer1 中斷標誌

```



```

118 void TMR1_IRQHandler(void)
149     key_buffer = temp_key; // 將按鍵值存入緩衝區
150     key_lock = 1;         // 鎖定按鍵，防止重複觸發
151 }
152 }
153 else
154 {
155     // 當 ScanKey 返回 0（無按鍵按下），解除鎖定
156     key_lock = 0;
157 }
158 }
159
160 TIMER_ClearIntFlag(TIMER1); // 清除 Timer1 中斷標誌
161 }
162
163 // ===== 6. Timer 初始化函數 =====
164 /**
165  * @brief 初始化 Timer0（動畫控制定時器）
166  * @details 設定 Timer0 為週期模式，頻率 100Hz（每 10ms 觸發一次中斷）
167  */
168 void Init_Timer0(void)
169 {
170     CLK_EnableModuleClock(TMR0_MODULE); // 啟用 Timer0 時鐘
171     CLK_SetModuleClock(TMR0_MODULE, CLK_CLKSEL1_TMR0_S_HXT, 0); // 設定時鐘源為 HXT（外部高速振盪器）
172     TIMER_Open(TIMER0, TIMER_PERIODIC_MODE, 100); // 設定為週期模式，頻率 100Hz
173     TIMER_EnableInt(TIMER0); // 啟用 Timer0 中斷
174     NVIC_EnableIRQ(TMR0_IRQn); // 在 NVIC 中啟用 Timer0 中斷
175     TIMER_Start(TIMER0); // 啟動 Timer0
176 }
177
178 /**
179  * @brief 初始化 Timer1（七段顯示器和按鍵掃描定時器）
180  * @details 設定 Timer1 為週期模式，頻率 1kHz（每 1ms 觸發一次中斷）
181  *          設定中斷優先權：Timer1 優先權 0（最高），Timer0 優先權 1
182  */
183 void Init_Timer1(void)
184 {
185     CLK_EnableModuleClock(TMR1_MODULE); // 啟用 Timer1 時鐘
186     CLK_SetModuleClock(TMR1_MODULE, CLK_CLKSEL1_TMR1_S_HXT, 0); // 設定時鐘源為 HXT
187     TIMER_Open(TIMER1, TIMER_PERIODIC_MODE, 1000); // 設定為週期模式，頻率 1kHz
188     TIMER_EnableInt(TIMER1); // 啟用 Timer1 中斷
189
190     // 設定中斷優先權（數值越小優先權越高）
191     NVIC_SetPriority(TMR1_IRQn, 0); // Timer1 優先權 0（最高，用於即時掃描）
192     NVIC_SetPriority(TMR0_IRQn, 1); // Timer0 優先權 1（較低）
193
194     NVIC_EnableIRQ(TMR1_IRQn); // 在 NVIC 中啟用 Timer1 中斷
195     TIMER_Start(TIMER1); // 啟動 Timer1
196 }

```

```

198 // ===== 7. LCD 動畫更新函數 =====
199 /**
200  * @brief 更新 LCD 顯示的動畫幀
201  * @details 清除 LCD 畫面並繪製當前動畫幀
202  */
203 void Update_Animation(void)
204 {
205     clear_LCD(); // 清除 LCD 畫面
206     // 在 LCD 中央位置 (x=32, y=0) 繪製 64x64 像素的點陣圖
207     draw_Bmp64x64(32, 0, FG_COLOR, BG_COLOR, (unsigned char *)frames[current_frame]);
208 }

210 // ===== 8. 按鍵處理函數 =====
211 /**
212  * @brief 處理按鍵輸入
213  * @details 從按鍵緩衝區讀取按鍵值並執行相應功能
214  *          注意：此版本 (Q1) 僅支援開始/停止功能，速度控制功能被註解
215  */
216 void Process_Keys(void)
217 {
218     uint8_t current_key;
219
220     // 從緩衝區讀取按鍵值
221     current_key = key_buffer;
222     key_buffer = 0; // 清除緩衝區
223
224     if (current_key != 0)
225     {
226         switch (current_key)
227         {
228             case 5: // 按鍵 S (開始/停止)
229                 if (is_running)
230                 {
231                     // 當前正在運行，執行停止操作
232                     is_running = 0; // 停止動畫
233                     time_counter = 0; // 重置時間計數器
234                     animation_counter = 0; // 重置動畫計數器
235                     seg_digit[0] = seg_digit[1] = seg_digit[2] = seg_digit[3] = 0; // 重置七段顯示器
236                     clear_LCD(); // 清除 LCD
237                     // 顯示紅色停止標誌
238                     draw_Bmp64x64(32, 0, FG_COLOR, BG_COLOR, (unsigned char *)red);
239                 }
240                 else
241                 {
242                     // 當前已停止，執行開始操作
243                     is_running = 1; // 啟動動畫
244                     time_counter = 0; // 重置時間計數器
245                     animation_counter = 0; // 重置動畫計數器
246                     current_frame = 0; // 從第一幀開始
247                     seg_digit[0] = seg_digit[1] = seg_digit[2] = seg_digit[3] = 0; // 重置七段顯示器
248                     lcd_update_flag = 1; // 設定 LCD 更新標誌
249                 }
250                 break;

```

```

216 void Process_Keys(void)
262     }
263 }
264 }
265
266 // ===== 9. 主程式 =====
267 /**
268  * @brief 主函數
269  * @details 系統初始化、硬體設定、主迴圈執行
270  */
271 int main(void)
272 {
273     // ===== 系統初始化 =====
274     SYS_Init(); // 系統初始化 (時鐘、GPIO 等)
275
276     // ===== 定時器初始化 =====
277     Init_Timer0(); // 初始化 Timer0 (動畫控制)
278     Init_Timer1(); // 初始化 Timer1 (掃描控制)
279
280     // ===== 週邊設備初始化 =====
281     OpenSevenSegment(); // 開啟七段顯示器
282     OpenKeyPad();       // 開啟按鍵矩陣
283
284     // ===== LCD 初始化 =====
285     init_LCD(); // 初始化 LCD
286     clear_LCD(); // 清除 LCD 畫面
287
288     // ===== 動畫幀陣列初始化 =====
289     frames[0] = green1; // 第 1 幀
290     frames[1] = green2; // 第 2 幀
291     frames[2] = green3; // 第 3 幀
292     frames[3] = green4; // 第 4 幀
293     frames[4] = green5; // 第 5 幀
294     frames[5] = green6; // 第 6 幀
295
296     // ===== 顯示初始畫面 =====
297     draw_Bmp64x64(32, 0, FG_COLOR, BG_COLOR, (unsigned char *)frames[0]);
298
299     // ===== 主迴圈 =====
300     while (1)
301     {
302         Process_Keys(); // 處理按鍵輸入
303
304         // 檢查是否需要更新 LCD (由 Timer0 中斷設定)
305         if (lcd_update_flag)
306         {
307             Update_Animation(); // 更新動畫顯示
308             lcd_update_flag = 0; // 清除更新標誌
309         }

```



```

271 int main(void)
282     OpenKeyPad();           // 開啟按鍵矩陣
283
284     // ===== LCD 初始化 =====
285     init_LCD(); // 初始化 LCD
286     clear_LCD(); // 清除 LCD 畫面
287
288     // ===== 動畫幀陣列初始化 =====
289     frames[0] = green1; // 第 1 幀
290     frames[1] = green2; // 第 2 幀
291     frames[2] = green3; // 第 3 幀
292     frames[3] = green4; // 第 4 幀
293     frames[4] = green5; // 第 5 幀
294     frames[5] = green6; // 第 6 幀
295
296     // ===== 顯示初始畫面 =====
297     draw_Bmp64x64(32, 0, FG_COLOR, BG_COLOR, (unsigned char *)frames[0]);
298
299     // ===== 主迴圈 =====
300     while (1)
301     {
302         Process_Keys(); // 處理按鍵輸入
303
304         // 檢查是否需要更新 LCD (由 Timer0 中斷設定)
305         if (lcd_update_flag)
306         {
307             Update_Animation(); // 更新動畫顯示
308             lcd_update_flag = 0; // 清除更新標誌
309         }
310     }
311 }

```

Lab10-2:

[illegible]


```
47 // 七段顯示器變數
48 volatile uint8_t seg_digit[4] = {0, 0, 0, 0}; // 七段顯示器四位數字陣列
49
50 // 速度設定陣列 (對應 speed_index 0-3)
51 // 數值越大，動畫切換越慢 (需要更多次 Timer0 中斷才切換一幀)
52 // speed_index 0: 200 (最慢), 1: 100, 2: 50, 3: 25 (最快)
53 const uint32_t speed_settings[4] = {200, 100, 50, 25};
54
55 // 動畫幀指標陣列 (指向 6 個綠色小人點陣圖)
56 const unsigned char *frames[6];
57
58 // ===== 3. 函數宣告 =====
59 void Init_Timer0(void);
60 void Init_Timer1(void);
61 void Update_Animation(void);
62 void Process_Keys(void);
```



```

64 // ===== 4. Timer0 中斷處理函數 (100Hz = 每 10ms 觸發一次) =====
65 /**
66  * @brief Timer0 中斷服務程式
67  * @details 負責控制動畫播放速度和時間計數
68  *          - 每 10ms 觸發一次 (100Hz)
69  *          - 根據速度設定決定何時切換動畫幀
70  *          - 更新時間計數器和七段顯示器數字
71  */
72 void TMR0_IRQHandler(void)
73 {
74     // 區域變數宣告 (C89 標準要求所有變數宣告必須放在函數開頭)
75     uint32_t total_seconds; // 總秒數 (用於計算七段顯示器的顯示值)
76
77     // 只有在動畫運行狀態下才執行以下操作
78     if (is_running)
79     {
80         // ===== 1. 動畫控制邏輯 =====
81         animation_counter++; // 動畫計數器遞增 (每次中斷 +1)
82
83         // 檢查是否達到當前速度設定值 (需要多少次中斷才切換一幀)
84         if (animation_counter >= speed_settings[speed_index])
85         {
86             animation_counter = 0; // 重置動畫計數器
87             current_frame = (current_frame + 1) % 6; // 切換到下一幀 (循環 0-5)
88             lcd_update_flag = 1; // 設定 LCD 更新標誌, 通知主程式更新畫面
89         }
90
91         // ===== 2. 時間計數 =====
92         time_counter++; // 時間計數器遞增 (每 10ms +1, 單位: 0.01秒)
93
94         // ===== 3. 數值轉換 (將時間計數器轉換為七段顯示器數字) =====
95         // 將時間計數器 (單位: 0.01秒) 轉換為總秒數
96         // 例如: time_counter = 1234 → total_seconds = 12.34 秒 (取整數部分)
97         total_seconds = time_counter / 100;
98
99         // 將總秒數分解為四位數字, 存入七段顯示器陣列
100        // seg_digit[0]: 個位數 (0-9)
101        seg_digit[0] = total_seconds % 10;
102        // seg_digit[1]: 十位數 (0-9)
103        seg_digit[1] = (total_seconds / 10) % 10;
104        // seg_digit[2]: 百位數 (0-9)
105        seg_digit[2] = (total_seconds / 100) % 10;
106        // seg_digit[3]: 千位數 (0-9)
107        seg_digit[3] = (total_seconds / 1000) % 10;
108    }
109
110    // 清除 Timer0 中斷標誌 (必須執行, 否則會持續觸發中斷)
111    TIMER_ClearIntFlag(TIMER0);
112 }

```

```

114 // ===== 5. Timer1 中斷處理函數（修正按鍵靈敏度版） =====
115 void TMR1_IRQHandler(void)
116 {
117     // 使用 static 變數保持狀態
118     static uint8_t scan_index = 0;
119     static uint8_t key_check_counter = 0;
120     static uint8_t prev_key = 0; // [新增] 用來記錄「上一次」的按鍵值
121     uint8_t temp_key;
122
123     // ===== 任務 1: 七段顯示器多工掃描（維持不變） =====
124     CloseSevenSegment();
125     ShowSevenSegment(scan_index, seg_digit[scan_index]);
126     scan_index++;
127     if (scan_index > 3) scan_index = 0;
128
129     // ===== 任務 2: 按鍵掃描（改良版） =====
130     key_check_counter++;
131
132     // [修改] 將檢查間隔從 20ms 縮短為 10ms，提高反應速度
133     if (key_check_counter >= 10) {
134         key_check_counter = 0;
135
136         temp_key = ScanKey(); // 讀取當前按鍵
137
138         // [核心修改] 去彈跳邏輯：
139         // 只有當「這次讀到的值」跟「上次讀到的值」一樣時，才視為穩定訊號
140         if (temp_key == prev_key) {
141
142             if (temp_key != 0) {
143                 // 確認是穩定的「按下」狀態
144                 if (key_lock == 0) {
145                     key_buffer = temp_key; // 寫入 Buffer
146                     key_lock = 1;          // 鎖定，避免長按重複觸發
147                 }
148             } else {
149                 // 確認是穩定的「放開」狀態（temp_key 為 0）
150                 key_lock = 0; // 解除鎖定，允許下一次按壓
151             }
152         }
153
154         // 更新歷史狀態，供下一次比較用
155         prev_key = temp_key;
156     }
157
158     TIMER_ClearIntFlag(TIMER1);
159 }

```

```

161 // ===== 6. Timer 初始化函數 =====
162 /**
163  * @brief 初始化 Timer0 (動畫控制定時器)
164  * @details 設定 Timer0 為週期模式，頻率 100Hz (每 10ms 觸發一次中斷)
165  */
166 void Init_Timer0(void)
167 {
168     CLK_EnableModuleClock(TMR0_MODULE); // 啟用 Timer0 時鐘
169     CLK_SetModuleClock(TMR0_MODULE, CLK_CLKSEL1_TMR0_S_HXT, 0); // 設定時鐘源為 HXT (外部高速振盪器)
170     TIMER_Open(TIMER0, TIMER_PERIODIC_MODE, 100); // 設定為週期模式，頻率 100Hz
171     TIMER_EnableInt(TIMER0); // 啟用 Timer0 中斷
172     NVIC_EnableIRQ(TMR0_IRQn); // 在 NVIC 中啟用 Timer0 中斷
173     TIMER_Start(TIMER0); // 啟動 Timer0
174 }
175
176 /**
177  * @brief 初始化 Timer1 (七段顯示器和按鈕掃描定時器)
178  * @details 設定 Timer1 為週期模式，頻率 1kHz (每 1ms 觸發一次中斷)
179  *          設定中斷優先權：Timer1 優先權 0 (最高)，Timer0 優先權 1
180  */
181 void Init_Timer1(void)
182 {
183     CLK_EnableModuleClock(TMR1_MODULE); // 啟用 Timer1 時鐘
184     CLK_SetModuleClock(TMR1_MODULE, CLK_CLKSEL1_TMR1_S_HXT, 0); // 設定時鐘源為 HXT
185     TIMER_Open(TIMER1, TIMER_PERIODIC_MODE, 1000); // 設定為週期模式，頻率 1kHz
186     TIMER_EnableInt(TIMER1); // 啟用 Timer1 中斷
187
188     // 設定中斷優先權 (數值越小優先權越高)
189     NVIC_SetPriority(TMR1_IRQn, 0); // Timer1 優先權 0 (最高，用於即時掃描)
190     NVIC_SetPriority(TMR0_IRQn, 1); // Timer0 優先權 1 (較低)
191
192     NVIC_EnableIRQ(TMR1_IRQn); // 在 NVIC 中啟用 Timer1 中斷
193     TIMER_Start(TIMER1); // 啟動 Timer1
194 }
195
196 // ===== 7. LCD 動畫更新函數 =====
197 /**
198  * @brief 更新 LCD 顯示的動畫幀
199  * @details 清除 LCD 畫面並繪製當前動畫幀
200  */
201 void Update_Animation(void)
202 {
203     clear_LCD(); // 清除 LCD 畫面
204     // 在 LCD 中央位置 (x=32, y=0) 繪製 64x64 像素的點陣圖
205     draw_Bmp64x64(32, 0, FG_COLOR, BG_COLOR, (unsigned char *)frames[current_frame]);
206 }

```



```

208 // ===== 8. 按鍵處理函數 =====
209 /**
210  * @brief 處理按鍵輸入
211  * @details 從按鍵緩衝區讀取按鍵值並執行相應功能
212  *          支援功能：
213  *          - 按鍵 S (5): 開始/停止動畫
214  *          - 按鍵 4: 減速 (降低動畫播放速度)
215  *          - 按鍵 6: 加速 (提高動畫播放速度)
216  */
217 void Process_Keys(void)
218 {
219     uint8_t current_key;
220
221     // 從緩衝區讀取按鍵值
222     current_key = key_buffer;
223     key_buffer = 0; // 清除緩衝區
224
225     if (current_key != 0)
226     {
227         switch (current_key)
228         {
229             case 5: // 按鍵 S (開始/停止)
230                 if (is_running)
231                 {
232                     // 當前正在運行，執行停止操作
233                     is_running = 0; // 停止動畫
234                     time_counter = 0; // 重置時間計數器
235                     animation_counter = 0; // 重置動畫計數器
236                     seg_digit[0] = seg_digit[1] = seg_digit[2] = seg_digit[3] = 0; // 重置七段顯示器
237                     clear_LCD(); // 清除 LCD
238                     // 顯示紅色停止標誌
239                     draw_Bmp64x64(32, 0, FG_COLOR, BG_COLOR, (unsigned char *)red);
240                 }
241                 else
242                 {
243                     // 當前已停止，執行開始操作
244                     is_running = 1; // 啟動動畫
245                     time_counter = 0; // 重置時間計數器
246                     animation_counter = 0; // 重置動畫計數器
247                     current_frame = 0; // 從第一幀開始
248                     seg_digit[0] = seg_digit[1] = seg_digit[2] = seg_digit[3] = 0; // 重置七段顯示器
249                     lcd_update_flag = 1; // 設定 LCD 更新標誌
250                 }
251                 break;

```

```
217 void Process_Keys(void)
253     case 4: // 按鍵 4 (減速)
254         if (speed_index > 0)
255         {
256             speed_index--; // 降低速度索引 (數值越小速度越慢)
257         }
258         animation_counter = 0; // 重置動畫計數器，立即套用新速度
259         break;
260
261     case 6: // 按鍵 6 (加速)
262         if (speed_index < 3)
263         {
264             speed_index++; // 提高速度索引 (數值越大速度越快)
265         }
266         animation_counter = 0; // 重置動畫計數器，立即套用新速度
267         break;
268     }
269 }
270 }
```

```

272 // ===== 9. 主程式 =====
273 /**
274  * @brief 主函數
275  * @details 系統初始化、硬體設定、主迴圈執行
276  */
277 int main(void)
278 {
279     // ===== 系統初始化 =====
280     SYS_Init(); // 系統初始化 (時鐘、GPIO 等)
281
282     // ===== 定時器初始化 =====
283     Init_Timer0(); // 初始化 Timer0 (動畫控制)
284     Init_Timer1(); // 初始化 Timer1 (掃描控制)
285
286     // ===== 週邊設備初始化 =====
287     OpenSevenSegment(); // 開啟七段顯示器
288     OpenKeyPad(); // 開啟按鍵矩陣
289
290     // ===== LCD 初始化 =====
291     init_LCD(); // 初始化 LCD
292     clear_LCD(); // 清除 LCD 畫面
293
294     // ===== 動畫幀陣列初始化 =====
295     frames[0] = green1; // 第 1 幀
296     frames[1] = green2; // 第 2 幀
297     frames[2] = green3; // 第 3 幀
298     frames[3] = green4; // 第 4 幀
299     frames[4] = green5; // 第 5 幀
300     frames[5] = green6; // 第 6 幀
301
302     // ===== 顯示初始畫面 =====
303     draw_Bmp64x64(32, 0, FG_COLOR, BG_COLOR, (unsigned char *)frames[0]);
304
305     // ===== 主迴圈 =====
306     while (1)
307     {
308         Process_Keys(); // 處理按鍵輸入
309
310         // 檢查是否需要更新 LCD (由 Timer0 中斷設定)
311         if (lcd_update_flag)
312         {
313             Update_Animation(); // 更新動畫顯示
314             lcd_update_flag = 0; // 清除更新標誌
315         }
316     }
317 }

```